

课程实验报告

网络安全学院

目 录

1 实验目的.....	1
2 实验环境.....	1
3 实验内容.....	1
3.1 安装、编译和初始化.....	1
3.2 密钥层次 (Key Hierarchy)	2
3.3 Seal、Unseal 和 extend.....	5
3.4 密钥迁移 (Key Migration)	9
3.5 远程证明 (Remote Attestation)	10
4 实验心得体会.....	13
5 实验代码.....	14
5.1 密钥层次创建 create_register_key.c.....	14
5.2 加载密钥 load_key.c.....	18
5.3 Unseal、解密文件 unseal_file.c.....	22
5.4 远程证明 platform_dst.c.....	26

1 实验目的

本实验的目的是让学生将从书本中学到的可信计算相关知识应用到实践中。在 linux 中使用 tpm 模拟器，通过 TSS 软件栈调用相关硬件来完成远程证明、密钥迁移、密钥结构、数据密封等相关功能，了解 TPM 的安全性，学会调用 TSS 的各种接口来完成应用程序。

本实验的任务主要是在随文档提供的代码的基础下，填补代码中缺失的部分，这个工作主要在密钥迁移和密钥结构相关功能代码中，还有根据功能需要，补全数据密封功能所需要的代码文件。

2 实验环境

- 1.Seed Ubuntu 12.04 LTS 32 位的 VMware 虚拟机
- 2.TPM Emulator
- 3.Trousers

3 实验内容

3.1 安装、编译和初始化

在 Ubuntu1204 虚拟机中，使用 TPM Emulator 完成 TPM 的相关实验，将 tpm-emulator.tar.gz 置于工作目录下，解压并编译安装 TPM Emulator。

```
tar xvf tpm-emulator.tar.gz
cd tpm-emulator
sudo apt-get install libgmp-dev cmake
./build.sh
cd build
sudo make install
sudo depmod -a
```

安装TSS软件栈：

```
sudo apt-get install libtspi-dev trousers
```

进入到 trusted-computing-projectv0.3 工作目录中，执行 make clean 并 make，编译实验代码。观察 trusted-computing-projectv0.3 目录结构，每个实验的目录中均有 makefile 文件，记录了该目录下源代码的配置信息，在父目录下执行 make，将根据父目录的 makefile 文件对每个实验目录进行遍历，根据相应 makefile 文件的配置信息进行编译。

进行初始化操作：

```
sudo modprobe tpmdev
sudo tpmdev -f -d clear
#再打开一个终端，不关闭前面tpmdev的终端
sudo tcscd
cd /home/seed/trusted-computing-projectv0.3/init
./Tspi_TPM_TakeOwnership01 -v 1.2
./create_mig_key -v 1.2
输入自定义pin
```

modprobe 载入特定模块 tpmdev，根据 depmod 所产生的相依关系，决定要载入哪些模块。若在载入过程中发生错误，在 modprobe 会卸载整组的模块。

调用 tpmdev 守护进程清理之前生成的数据。

启动 tcscd。Tcscd 是一个用户空间下的守护进程，打开 TPM 设备驱动程序，所有对 TPM 的请求都应该通过 TSS 堆栈。tcscd 管理 TPM 资源并处理来自 TSP 的本地和远程请求。

初始化 TPM，take ownership 获得用户身份，生成 AIK。之后，只有 TPM owner 能访问 EK，这一步往 TPM 中加入 PIN，并生成 SRK。

3.2 密钥层次 (Key Hierarchy)

TPM 对密钥进行分层管理，下层密钥由上层密钥进行加密保护。SRK 是存储根密钥，保护了所有的密钥，由于 SRK 的私钥始终位于 TPM 芯片内部，保护了密钥的安全性。

加载密钥时，要先将其父密钥加载进 TPM 中，用父密钥解密该密钥，这体现了密钥的层次结构。由于 TPM 密钥分为可迁移密钥和不可迁移密钥，其中不可迁移密钥可以保护可迁移密钥，而可迁移密钥只能保护可迁移密钥。可迁移密钥和不可迁移密钥中又分为存储密钥，签名密钥，绑定密钥等，他们根据 TPM 的密钥层次结构，由相应的父密钥进行保护。

本次实验需要构建的密钥层次结构如图 3.2.1 所示，SRK 作为存储根密钥，位于 TPM 芯片内部，保护着所有的密钥。K1 为不可迁移的存储密钥，保护不可迁移的签名密钥 K2；K3 为可迁移的存储密钥，保护可迁移绑定密钥 K4。阅读实验代码 create_register_key.c 发现，K1、K2、K3 的生成以及层次结构样例代码已经给出，实验需要完成 K4 的生成并使 K3 成为 K4 的父密钥。

代码中有一些固定的函数用于初始化环境，TSPI 为 TCG 软件栈提供的应用程序接口，通过调用 TSPI 接口函数来访问 TPM emulator，实验中所有对 TPM 的操作均通过 TSPI 接口函数实现。

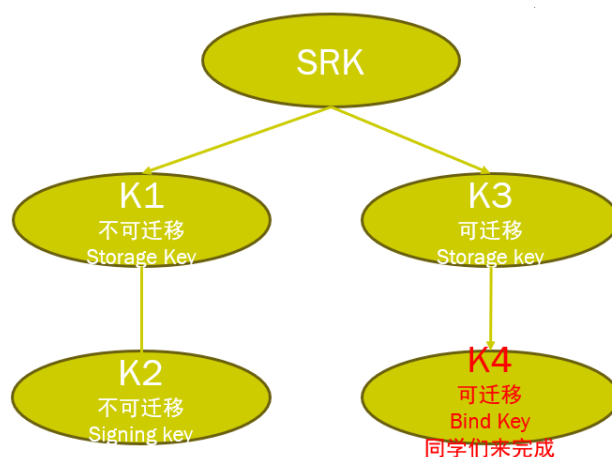


图 3.2.1 密钥层次结构

其中 `Tspi_Context_Create` 用于创建上下文环境，上下文句柄由变量 `hContext` 保存；`Tspi_Context_Connect` 建立起上下文与 TPM 的连接；`Tspi_Context_LoadKeyByUUID` 在上下文环境中，通过密钥的 UUID 加载密钥，获得相应的密钥句柄。根据 TPM 密钥层次结构，首先加载 SRK，故使用 `SRK_UUID` 加载 SRK，存于 `hSRK` 变量中；`Tspi_GetPolicyObject` 用于获得对象策略，该函数判断对象类型，加载对象策略，实验中首先加载 SRK 的策略，存于 `srkUsagePolicy` 中；`Tspi_Policy_SetSecret` 根据策略设置秘密信息。

为防止密钥重复注册，生成密钥层次前，把之前的密钥取消注册，调用 `Tspi_Context_UnregisterKey`，分别注销 K1、K2、K3、K4；之后重新生成，自行编写函数 `my_create_key`，该函数用于生成 TPM 密钥，再编写函数 `my_create_load_key` 用于加载密钥。`my_create_load_key` 检查密钥句柄对应的密钥是否已经存在，若存在则调用 `Tspi_Key_LoadKey` 根据其父密钥加载；若不存在则调用 `my_create_key` 生成密钥。

由于先前已经对所有密钥取消注册，故调用 `my_create_load_key` 时均为生成密钥，代码中 K1、K2、K3 的生成和注册已经实现，故只需生成和注册 K4。调用 `my_create_load_key` 生成 K4，通过参数 `initFlags` 向 `my_create_load_key` 传递 K4 的相关信息；调用 `Tspi_Context_RegisterKey` 注册 K4，根据 UUID 指定 K4 的父密钥为 K3 即可。

在 `trusted-computing-projectv0.3/KeyHierarchy` 目录下，根据以上分析并参考 K1、K2、K3 的创建过程，以及 TSS 文档，完善 `create_register_key.c` 中创建 K4 的代码，编译并运行：

```
make
./create_register_key -v 1.2
```

运行结果如图 3.2.2 所示

```

<<<test_start>>>
Testing Create KEY
KEY structure:
SRK
|__UserK1(Storage key, unmigratable)
|  |__UserK2(Signing key, unmigratable)
|
|__UserK3(Storage key, migratable)
|  |__UserK4(Bind key, migratable)

Create UserK1 and register it to disk.
Input K1's Usage Pin
Enter PIN:
Verifying - Verify PIN:
Create and register K1 succeeded!
Create UserK2 and register it to disk.
Input K2's Usage Pin
Enter PIN:
Verifying - Verify PIN:
Create and register K2 succeeded!
Create UserK3 and register it to disk.
Input K3's Usage Pin
Enter PIN:
Verifying - Verify PIN:
^^migratable
K3's Migration Pin
Enter PIN:
Verifying - Verify PIN:
Create and register K3 succeeded!
Create UserK4 and register it to disk.
Input K4's Usage Pin
Enter PIN:
Verifying - Verify PIN:
^^migratable
K4's Migration Pin
Enter PIN:
Verifying - Verify PIN:
Create and register K4 succeeded!
1 PASS : Create KEY returned (0) TSS_SUCCESS
Cleaning up Create KEY
<<<end_test>>>

```

图 3.2.2 密钥生成和注册运行结果

由以上的分析可知，加载密钥时需要先加载父密钥，阅读 load_key.c 发现加载 SRK、K1、K2、K3 的代码已经实现，根据样例代码加载 K4。调用 Tspi_Context_GetKeyByUUID 获得 K4 的 UUID，调用 set_popup_secret 获取用户输入的 PIN，验证使用策略；若用户输入了正确的 PIN，调用 Tspi_Key_LoadKey，传入 hKey4、hKey3 密钥句柄，加载 K4。

编译运行 load_key，./load_key -v 1.2，实验结果如图 3.2.3 所示。

```

<<<test_start>>>
Testing Load KEY
KEY structure:
SRK
|__UserK1(Storage key, unmigratable)
|  |__UserK2(Signing key, unmigratable)
|
|__UserK3(Storage key, migratable)
|  |__UserK4(Bind key, migratable)

Load SRK sucesed!
Loading K1...
Load UserK1 sucesed!
Loading K2...
Input K1's pin
Enter PIN:
Load UserK2 sucesed!
Loading K3...
Load UserK3 sucesed!
Loading K4...
Load UserK4 sucesed!
Input K3's pin
Enter PIN:
Load UserK4 sucesed!
1 PASS : Load KEY returned (0) TSS_SUCCESS
Cleaning up Load KEY
<<<end_test>>>

```

图 3.2.3 加载密钥

3.3 Seal、Unseal 和 extend

TPM 提供 Seal/Unseal 机制，可以通过将秘密信息绑定 TPM 特定的 PCR 值实现数据的保护，只有特定 PCR 寄存器满足一定值时才能解封秘密信息。TCG 软件栈提供了 Seal/Unseal 的编程接口，应用层可调用这些 API 实现秘密信息的封装和解封。

阅读样例代码 seal.c、unseal.c、seal_file.c 发现 seal.c、unseal.c 已经实现了秘密数据封装和解封的样例代码，seal_file.c 实现了将对称密钥 Seal，并加密文件保存在密文文件中的功能。实验根据以上 3 个文件，自行编写 unseal_file.c，实现从密文文件中提取对称密钥，并解密文件获得明文的过程。

在 seal_file.c 中，首先生成上下文环境，加载密钥，获得密钥策略等；调用 Tspi_TPM_GetRandom 通过 TPM 的随机数发生器，生成随机数 random，random 作为对称加密密钥用于对文件加密；调用 Tspi_TPM_PcrRead 获得 PCR 值，存于 rgbPcrVal 变量中，实验中获得 PCR15 的值；调用 Tspi_PcrComposite_SetPcrValue 将 PCR15 的值与 PCR 句柄 hPcrs 绑定。

调用 Tspi_Data_Seal 进行 Seal 操作，用 K1 加密对称密钥 random，与 PCR15 绑定后封装，获得句柄 hEncData；调用 Tspi_GetAttribData 获得 hEncData 中秘密数据的长度和秘密数据的值，分别存于 u32EncDataLen 和 rgbEncData 中；至此已经将对称密钥封装后的相关信息准备好。

接下来读入文件，获得文件长度 size，把文件内容读入到缓冲区 pBufIn 中；调用 aes_init

初始化 AES 加密，调用 `aes_encrypt` 将 `pBufIn` 中的数据用 `random` 加密，写入到 `pBufOut` 中。打开密文文件，向其中分别写入 `u32EncDataLen`、`rgbEncData`、`size`、`pBufOut`，分别是封装信息的长度，封装的秘密信息，文件长度和文件密文。

编写 `unseal_file.c` 时，根据封装加密的过程，构造逆过程即可。根据 `aes_encrypt` 构造 AES 解密函数 `aes_decrypt`，TPM 初始化的部分不变，不过不需要再获得 PCR 的值。读文件，首先读 4 字节数据作为 32 位整型数据存于 `u32EncDataLen`，前 4 字节为封装秘密数据的长度；根据 `u32EncDataLen` 的值读取相应长度的封装秘密数据，存于 `rgbEncData`；再读 4 字节数据作为 32 为整型存于 `size`，为密文长度；根据 `size` 值读取密文存于 `pBufIn`。

调用 `Tspi_SetAttribData` 将封装秘密数据读取出来，获得秘密数据句柄 `hEncData`；调用 `Tspi_Data_Unseal` 将 `hEncData` 所指的秘密数据解封，若 PCR 满足相应状态，则能成功解封，解封的数据长度存于变量 `DataUnsealedLen`，数据值存于 `DataUnsealed`，若解封成功，则 `DataUnsealed` 即为 AES 对称密钥。

调用 `aes_init` 初始化 AES，调用 `aes_decrypt` 解密数据获得明文，存于 `pBufOut`，将 `pBufOut` 写入文件。

在 `SealUnseal` 目录中，编译执行以上代码：

`./seal -v 1.2` （成功）

```
[06/08/2020 04:16] seed@ubuntu:~/trust/trusted-computing-projectv0.3/SealUnseal$ ./seal -v 1.2
1
2
3
4
5
6
7
EncDataBlob:
00000000| 01 01 00 00 00 00 00 2c 00 02 00 80 c6 58 2c 81 .....X,..
00000010| 00 7f 85 4e d3 87 3c 59 63 7a 01 8a 44 19 63 97 ...N...<Ycz..D.C.
00000020| c6 58 2c 81 00 7f 85 4e d3 87 3c 59 63 7a 01 8a .X,...N...<Ycz..
00000030| 44 19 63 97 00 00 01 00 0d a6 20 49 6e b3 74 fe D.c.....In.t.
00000040| aa ed 2f 75 2e 51 a0 fe 44 8f c7 39 38 e6 92 47 ..u.Q..D..98..G
00000050| 2b 75 7b 4c 9e 28 6c d4 63 57 d9 1b af 27 09 4c +u{L.(l.cw...'.L
00000060| c7 c2 3d 12 f2 14 34 0b 4c a1 f0 c5 20 4a 85 a9 ..=...4.L...J..
00000070| 2f d7 65 ea 54 c6 19 df 9d 0d d5 33 2e 44 a1 17 /.e.T.....3.D..
00000080| 6c 6e c8 6e 7a 28 39 5b f9 ad 14 7d 02 74 bf 8e ln.nz(9[...].t..
00000090| ac 0d 40 6b 3a d0 c7 c3 3c 28 0a c7 5d 63 c9 a1 ..@k:...<[...]c..
000000a0| 9e 0f d2 f5 aa 59 7b 7a 27 c9 5a 87 23 7a 1a 73 .....Y{z'.Z.#z.s
000000b0| e1 0d ac e6 ac 8c 07 eb 02 45 a4 0a 15 ea 57 73 .....E....WS
000000c0| 60 1c 83 eb ce a2 28 0e 2f 4b 16 1a 2c b2 54 91 `.....(/K...T.
000000d0| 71 59 34 5e 55 b7 ad 67 6f 73 88 66 be 02 dc 36 qY4^U..gos.f...6
000000e0| 9c d9 f6 31 26 c2 84 f9 cd 96 5e 31 a7 83 a9 22 ...1&.....^1..."
000000f0| a5 f7 c7 af ec 89 ed e8 c2 dd 43 72 ac 62 69 e6 .....Cr.bl.
00000100| b7 f3 7f 0f 2b 90 d8 38 13 21 ef 25 df be 93 8e ....+..8.!%....
00000110| 5e e7 29 a5 dc e4 48 46 4f e4 78 44 ca 45 9c c1 ^.)...HFO.xD.E..
00000120| 69 23 e4 b3 d2 67 43 72 bc d2 b7 19 54 47 e6 ba t#...gCr....TG..
00000130| da 0d 4c 69 56 9f da 5b ...LtV..[
Success
```

图 3.3.1 `./seal -v 1.2`

`./unseal -v 1.2` （成功）

```
[06/08/2020 04:17] seed@ubuntu:~/trust/trusted-computing-projectv0.3/SealUnseal$ ./unseal -v 1.2
Sealed data:
00000000| 01 01 00 00 00 00 00 2c 00 02 00 80 c6 58 2c 81 .....X,.
00000010| 00 7f 85 4e d3 87 3c 59 63 7a 01 8a 44 19 63 97 ...N.<Ycz..D.C.
00000020| c6 58 2c 81 00 7f 85 4e d3 87 3c 59 63 7a 01 8a .X,...N.<Ycz..
00000030| 44 19 63 97 00 00 01 00 0d a6 20 49 6e b3 74 fe D.C..... In.t.
00000040| aa ed 2f 75 2e 51 a0 fe 44 8f c7 39 38 e6 92 47 ../u.Q..D..98..G
00000050| 2b 75 7b 4c 9e 28 6c d4 63 57 d9 1b af 27 09 4c +u{L.(l.cw...'.L
00000060| c7 c2 3d 12 f2 14 34 0b 4c a1 f0 c5 20 4a 85 a9 ..=...4.L... J..
00000070| 2f d7 65 ea 54 c6 19 df 9d 0d d5 33 2e 44 a1 17 /.e.T.....3.D..
00000080| 6c 6e c8 6e 7a 28 39 5b f9 ad 14 7d 02 74 bf 8e ln.nz(9[...].t..
00000090| ac 0d 40 6b 3a d0 c7 c3 3c 28 0a c7 5d 63 c9 a1 ..@k:...<[.]c..
000000a0| 9e 0f d2 f5 aa 59 7b 7a 27 c9 5a 87 23 7a 1a 73 .....Y{z'.Z.#z.s
000000b0| e1 0d ac e6 ac 8c 07 eb 02 45 a4 0a 15 ea 57 73 .....E....Ws
000000c0| 60 1c 83 eb ce a2 28 0e 2f 4b 16 1a 2c b2 54 91 `.....(/K...,T.
000000d0| 71 59 34 5e 55 b7 ad 67 6f 73 88 66 be 02 dc 36 qY4^U..gos.f...6
000000e0| 9c d9 f6 31 26 c2 84 f9 cd 96 5e 31 a7 83 a9 22 ...18.....^1..."
000000f0| a5 f7 c7 af ec 89 ed e8 c2 dd 43 72 ac 62 69 e6 .....Cr.bi.
00000100| b7 f3 7f 0f 2b 90 d8 38 13 21 ef 25 df be 93 8e ....+..8.!.%....
00000110| 5e e7 29 a5 dc e4 48 46 4f e4 78 44 ca 45 9c c1 ^.)...HFO.xD.E..
00000120| 69 23 e4 b3 d2 67 43 72 bc d2 b7 19 54 47 e6 ba i#...gCr....TG..
00000130| da 0d 4c 69 56 9f da 5b ..LlV..[
Unsealed Data:
00000000| 30 31 32 33 34 35 36 37 38 39 41 42 43 44 45 46 0123456789ABCDEF
00000010| 30 31 32 33 34 35 36 37 38 39 41 42 43 44 45 46 0123456789ABCDEF
Success
```

图 3.3.2 ./unseal -v 1.2

#扩展 PCR 寄存器

./extend -v 1.2 (成功)

```
[06/08/2020 04:17] seed@ubuntu:~/trust/trusted-computing-projectv0.3/SealUnseal$ ./extend -v 1.2
ulPcrValLen:20
Success
```

图 3.3.3 ./extend -v 1.2

./unseal -v 1.2 (失败)

```
[06/08/2020 04:18] seed@ubuntu:~/trust/trusted-computing-projectv0.3/SealUnseal$ ./unseal -v 1.2
Sealed data:
00000000| 01 01 00 00 00 00 00 2c 00 02 00 80 c6 58 2c 81 .....X,.
00000010| 00 7f 85 4e d3 87 3c 59 63 7a 01 8a 44 19 63 97 ...N.<Ycz..D.C.
00000020| c6 58 2c 81 00 7f 85 4e d3 87 3c 59 63 7a 01 8a .X,...N.<Ycz..
00000030| 44 19 63 97 00 00 01 00 0d a6 20 49 6e b3 74 fe D.C..... In.t.
00000040| aa ed 2f 75 2e 51 a0 fe 44 8f c7 39 38 e6 92 47 ../u.Q..D..98..G
00000050| 2b 75 7b 4c 9e 28 6c d4 63 57 d9 1b af 27 09 4c +u{L.(l.cw...'.L
00000060| c7 c2 3d 12 f2 14 34 0b 4c a1 f0 c5 20 4a 85 a9 ..=...4.L... J..
00000070| 2f d7 65 ea 54 c6 19 df 9d 0d d5 33 2e 44 a1 17 /.e.T.....3.D..
00000080| 6c 6e c8 6e 7a 28 39 5b f9 ad 14 7d 02 74 bf 8e ln.nz(9[...].t..
00000090| ac 0d 40 6b 3a d0 c7 c3 3c 28 0a c7 5d 63 c9 a1 ..@k:...<[.]c..
000000a0| 9e 0f d2 f5 aa 59 7b 7a 27 c9 5a 87 23 7a 1a 73 .....Y{z'.Z.#z.s
000000b0| e1 0d ac e6 ac 8c 07 eb 02 45 a4 0a 15 ea 57 73 .....E....Ws
000000c0| 60 1c 83 eb ce a2 28 0e 2f 4b 16 1a 2c b2 54 91 `.....(/K...,T.
000000d0| 71 59 34 5e 55 b7 ad 67 6f 73 88 66 be 02 dc 36 qY4^U..gos.f...6
000000e0| 9c d9 f6 31 26 c2 84 f9 cd 96 5e 31 a7 83 a9 22 ...18.....^1..."
000000f0| a5 f7 c7 af ec 89 ed e8 c2 dd 43 72 ac 62 69 e6 .....Cr.bi.
00000100| b7 f3 7f 0f 2b 90 d8 38 13 21 ef 25 df be 93 8e ....+..8.!.%....
00000110| 5e e7 29 a5 dc e4 48 46 4f e4 78 44 ca 45 9c c1 ^.)...HFO.xD.E..
00000120| 69 23 e4 b3 d2 67 43 72 bc d2 b7 19 54 47 e6 ba i#...gCr....TG..
00000130| da 0d 4c 69 56 9f da 5b ..LlV..[
0 FAIL : Tspi_Data_Unseal returned (24) TPM_E_WRONGPCRVAL
unseal.c 0 FAIL : Tspi_Data_Unseal returned (24) TPM_E_WRONGPCRVAL
```

图 3.3.4 ./unseal -v 1.2

./seal_file test.c test.en (查看文件 test.en 的内容)

```
[06/08/2020 04:18] seed@ubuntu:~/trust/trusted-computing-projectv0.3/SealUnseal$ ./seal_file test.c test.
en
Input K1's Pin
Enter PIN:
.....
```




图 3.3.8 `./unseal_file test.en test.de`

3.4 密钥迁移 (Key Migration)

本实验需要 2 个虚拟机，可将原来的 虚拟机拷贝一份，也可以用一个新的虚拟机重新配置环境。根据实验指导书，假定密钥从机器二迁移到机器一，则机器一需要将自己的公钥封装成文件发送给机器二，机器二将待迁移密钥的父密钥转为机器一发来的密钥，机器二将转换后的文件发送给机器一，机器一经过私钥解密后得到了迁移密钥。

阅读样例代码发现，已经实现了机器二将转换后的文件发送给机器一，实验要继续实现机器一获得迁移密钥的过程。已经从 `mig.blob` 中已经获取了随机值长度 `u32RandomLen`，随机值参数 `pRandom`，迁移块长度 `u32MigBlobLen`，迁移块数据 `pMigBlob`，参考 `Tspi_Key_ConvertMigrationBlob`，从迁移块中获取迁移密钥句柄 `hNewMigKey`；由于在第一步发给机器一的是机器二 SRK 的公钥，则此时迁移密钥的父密钥为 SRK。调用 `Tspi_Key_LoadKey` 加载迁移密钥；调用 `sign_and_verify` 在上下文中对迁移密钥进行签名和验证，完成密钥的迁移。

在 2 个机器中分别编译运行代码，在机器一中运行 `./platform_dst -g`，产生公钥文件 `srk.pub`

```
[06/08/2020 06:04] seed@ubuntu:~/trust/trusted-computing-projectv0.3/KeyMigration$ ./platform_dst -g
Generating Pub Key Success
```

图 3.4.1 `./platform_dst -g`

把文件 `srk.pub` 拷贝到机器二中，运行 `./platform_src`

```
[06/08/2020 06:12] seed@ubuntu:~/trust/trusted-computing-projectv0.3/KeyMigration$ ./platform_src
Input Migration Key's Pin

Enter PIN:
OK
```

图 3.4.2 ./platform_src

产生名为 mig.blob 的文件，文件 mig.blob 拷回到机器一中，在机器一中运行 ./platform_dst -m。

```
[06/08/2020 06:13] seed@ubuntu:~/trust/trusted-computing-projectv0.3/KeyMigration$ ./platform_dst -m
Data before signing:
30 39 38 37 36 35 34 33 32 31 30 39 38 37 36 35
34 33 32 31
Data after signing:
72 e8 c9 2f ec b8 50 12 a9 c8 1c 47 9e 6e 6d 28
e6 40 e2 b9 2d e0 b7 4c 9b 2d fa b1 21 8f fe de
c5 f1 53 d5 b9 81 4f 3c 09 c7 37 f7 e5 bf 7e ee
53 70 55 d5 af 45 d7 1d 3c 62 74 6c 07 ed 7f 75
51 4a 45 44 44 8d 02 6e 98 45 24 cc be 5b 87 31
51 98 ec 92 a3 6b b3 47 88 d3 f9 8a 23 91 37 1f
98 e5 71 3e 91 c0 71 19 c2 08 5a 81 ce f4 28 93
94 9f c8 9b de af 5e 19 2f ba 86 a0 1a ba 6a 4c
6d 3a 87 56 4a 57 2b 9d 48 25 6f 0e 40 f5 82 ba
f6 ce a5 53 ea 1a 64 02 2c e9 0d e5 70 50 cd da
a0 91 46 d3 60 b3 c8 6e 8e 95 f5 d3 38 fa 76 eb
c5 c6 2c af 8d 0a 8b 4e 1d fc 15 5e f0 a8 0f 1e
fb 8a 99 14 51 9a d5 0c 92 78 db 89 bc 1f db 99
52 3e 6e 5c a7 8e a1 f4 99 6d 3f 15 02 dc c5 29
a2 f7 f4 80 37 2f ab 43 28 8a 08 52 3e c8 3d 31
30 ad ba 21 1d 5c a0 4d 34 c3 f5 57 68 7d 89 c8
Data after verifying:
30 39 38 37 36 35 34 33 32 31 30 39 38 37 36 35
34 33 32 31
Migration Success
```

图 3.4.3 ./platform_dst -m

3.5 远程证明 (Remote Attestation)

本实验可继续使用上一个实验的 2 台虚拟机，远程证明过程如图 3.5.1 所示。

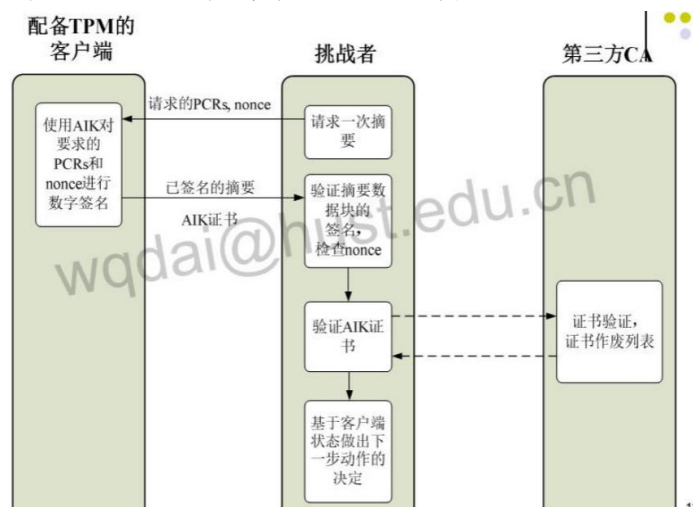


图 3.5.1 远程证明过程

配置虚拟机网络，使 2 台虚拟机可互通。根据实验指导书，假定机器二为挑战者，验证机器一，则在机器一中进入 RemoteAttestation/init 中执行 ./Create_AIK;

```
[06/08/2020 06:16] seed@ubuntu:~/trust/trusted-computing-projectv0.3/RemoteAttestation/init$ ./Create_AIK
497 Beginning remote_attestation()
[Client:]Tspi_TPM_CollateIdentityRequest.....
[CA:]Decrypt symmetric key.....
[CA:]Decrypt symmetric blob
[CA:] Verify the identity binding.....
[CA:]Create a fake credential and encrypt it.....
[Client:]Activate the TPM identity, receiving back the decrypted credential.....
Create_AIK.c:762 receive_msg_size is 44
Create_AIK.c:780          1 PASS : Create AIK returned (0) TSS_SUCCESS
786 Ending remote_attestation()
801 remote_data_size 44
803 keyPubSize 256
Create_AIK.c:805 ##The content of the keyPub is:##
af 20 63 f9 5c d8 ff e5 df 8c f1 6b fc 14 71 8a
71 88 07 ff 15 74 05 f4 0a ca 18 22 a6 74 eb db
7e 91 a5 56 e5 4c bc e0 62 51 17 9e 2a b1 32 b6
bd 18 7b 77 57 47 48 a4 c4 fe 49 8d 93 5a 3c d2
15 02 f7 4d b0 a7 f5 ad cd 6e fd c6 de 9b 8c 6d
5c 8c c8 cc 3f f7 49 75 34 fd ba c0 1f 2f 82 45
54 0f 72 be ab dd 1f 25 9c 5c 60 73 4b ab 65 b4
7f a2 26 21 a9 c3 07 70 c0 41 6f da 2d 1e ed 2c
ed bf a7 1b 29 75 f2 7c 63 2a 77 2e d3 ac b3 56
3e 3a 73 fe 03 40 15 fb 43 ea 66 d8 87 da 57 49
ed 2a f4 07 c2 f7 db 88 26 04 44 27 78 1a 09 45
24 45 aa 70 d3 41 dd f1 14 73 38 a6 49 16 7b bb
4c 53 fb 4b f9 45 dc d6 16 c2 19 54 e2 94 33 0c
73 59 20 1f c9 4f c7 09 13 95 fa 49 09 6e c5 69
c8 11 dd 8c 80 51 df 85 25 fd db a5 e7 3b 9f 0a
aa e7 3c b0 06 3f 9b 76 25 23 bc 1d 42 23 8b d1
Create_AIK.c:807 ##The content of the keyPub is over##
809 credLen 104
Create_AIK.c:811 ##The content of the cred is:##
5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a
5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a
5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a 5a
```

图 3.5.1 ./Create_AIK

返回到父目录，执行 ./RAServer

```

00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00
remote_attestation.c:842 ##The content of the pcr14 Value is over##
remote_attestation.c:850 selectpcrSize: 80 nr_select_PCR: 4
Tspi_TPM_Quote...
remote_attestation.c:956 pcr_tpmSize: 80
remote_attestation.c:969 requested malloc size for rc = 1032
remote_attestation.c:992 *****0*****
remote_attestation.c:992 *****0*****
remote_attestation.c:992 *****0*****
remote_attestation.c:992 *****0*****
remote_attestation.c:992 *****0*****
remote_attestation.c:992 *****0*****
remote_attestation.c:992 *****0*****
remote_attestation.c:992 *****1*****
remote_attestation.c:992 *****2*****
remote_attestation.c:992 *****0*****
remote_attestation.c:992 *****0*****
remote_attestation.c:992 *****16*****
remote_attestation.c:992 *****0*****
remote_attestation.c:992 *****64*****
remote_attestation.c:992 *****0*****
remote_attestation.c:995 pcr_tpmSize = 80
pcr_tpm:
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
remote_attestation.c:1005 receive_msg_size is 664
remote_attestation.c:1181 1 PASS : Remote Attestation returned (0) TSS_SUCCESS
1187 remote_data_size: 24 receive_msg_size: 664
1188 Ending remote_attestation()
##The nonce received from the client is:##
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
10 11 12 13
##The nonce received from the client is over##
===== Send Encrypted PCR to RAClient =====

```

图 3.5.2 ./RAServer

在机器二中进入 RemoteAttestation，执行./RAClient 机器 2 的 ip 机器 1 的 ip

```

tation$ ./RAClient 192.168.100.2 192.168.100.131
source IP: 192.168.100.131###
--> --> Dealing with No.1: 192.168.100.131===== Start to Communicate with RASer
ver =====
#####type: 2
===== Check Status DONE: CONNECTION Permitted! =====
sizeof_rc=====88

keyPubSize=====256

===== The length of the PCR response(buffer) is: 664 =====
RAClient.c:433 ##The content of the PCR response buffer is:##
30 00 00 00 00 01 00 00 00 01 00 00 58 00 00 00
01 01 00 00 51 55 4f 54 10 ea a7 90 83 cb 30 d3
d9 86 6a ad 27 52 f9 33 2d 82 a4 ec 00 01 02 03
04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 11 12 13
90 c0 8a cc 93 87 f7 51 a5 7a 7f 7a b9 8e 5d ff
9d 95 b8 eb 9a 10 b8 fa 5f 38 8c 1d c5 8f ac 81
73 b8 20 50 91 0e 26 e1 4e c1 4f 40 5d 54 83 e9
dd 70 72 30 47 df c6 ac d8 dd 4d cb b5 f5 61 b7
36 bb f3 37 56 82 15 e1 dc 82 5c ad 2c b4 7a 17
6c d2 31 08 a1 2c 40 cd 1e 15 bb 12 1f 8a e3 9e

```

图 3.5.3 ./RAClient 机器 2 的 ip 机器 1 的 ip

4 实验心得体会

通过本次实验，我对 TPM 可信计算模块的编程有了亲身的实践和体会，借助 TPM Emulator 模拟真实的 TPM 硬件环境，实现了一定层次的密钥生成和加载，Seal/Unseal，密钥迁移和远程证明。通过本次实验，我加深了对 TPM 核心功能的理解，体会到可信计算模块在实际应用中的重要作用，熟悉了基于 Linux 平台的可信计算程序编程，了解了 TCG 软件栈和 TPM 编程环境配置，对今后的学习和实践积累了一定的经验。

在实验过程中，我遇到了很多问题，进展并不顺利。在阅读样例代码时，我难以迅速理解代码对应的功能，发现自己对理论知识的理解存在很多欠缺。在实验时我还要经常翻看课件回顾所学知识，说明自己对知识的掌握仍不够熟练。在仔细对相关知识进行复习，查阅相关资料后，我逐渐了解了基于 TCG 软件栈的可信计算程序编写的大致流程，其中创建上下文，连接上下文，加载密钥，获得策略等是生成 TPM 可信环境的固定步骤。之后的不同功能继续调用相应的 API 接口实现。

实验中遇到困难最多的是 Seal/Unseal 部分，由于自己对 Seal/Unseal 的理论知识理解不够深刻，我又翻看了课件回顾所学知识。由于这个实验的代码比较多，理解代码也花了一些时间，在编写 unseal_file.c 的过程中，对密文文件的结构没有清晰的认识，导致读文件出现了错误。在仔细阅读 seal_file.c 后，我确定了写入文件的详细内容，完成了该部分实验。

可信计算实验是我第一次编写基于 TPM 的可信程序代码，是编写可信计算程序最初的尝试。仅靠本次实验远远达不到清楚掌握 TCG 软件栈，编写可靠程序的要求。在今后的学习中，我要继续巩固所学知识，将所学知识灵活应用于学习和实践中。脚踏实地学习理论知识，扎实进行实践环节，积极向同学老师请教交流，不断学习更多知识。

由于可信计算课程和实验是在抗击新冠肺炎疫情期间在家中完成的，学习和实验上存在各种困难，我通过请教老师和助教，与同学进行交流，查询大量资料逐一解决了问题。在此感谢在疫情期间仍为我们默默付出的老师们。

5 实验代码

5.1 密钥层次创建 create_register_key.c

```
#include "common.h"

TSS_RESULT
my_create_key(TSS_HCONTEXT hContext, TSS_FLAG initFlags, TSS_HKEY hParent, TSS_HKEY *hKey, const char
*pName);

/* create and load a key off the parentKey */
TSS_RESULT
my_create_load_key(TSS_HCONTEXT hContext, TSS_FLAG initFlags,
                    TSS_HKEY hParent, TSS_HKEY *hKey, const char *pName);
char *key_structure = "SRK\n"
    "__UserK1(Storage key, unmigratable)\n"
    "| __UserK2(Signing key, unmigratable)\n"
    "| \n"
    "    __UserK3(Storage key, migratable)\n"
    "    | __UserK4(Bind key, migratable)\n";
TSS_UUID UUID_K1 = {0, 0, 0, 0, 0, {8, 0, 0, 0, 0, 1}};
TSS_UUID UUID_K2 = {0, 0, 0, 0, 0, {8, 0, 0, 0, 0, 2}};
TSS_UUID UUID_K3 = {0, 0, 0, 0, 0, {8, 0, 0, 0, 0, 3}};
TSS_UUID UUID_K4 = {0, 0, 0, 0, 0, {8, 0, 0, 0, 0, 4}};
int main(int argc, char **argv)
{
    char version;

    version = parseArgs(argc, argv);
    if (version)
        main_v1_1();
    else
        print_wrongVersion();
}
main_v1_1(void)
{
    char *nameOfFunction = "Create KEY";
    TSS_HCONTEXT hContext;
    TSS_HTPM hTPM;
    TSS_FLAG initFlags;
    TSS_HKEY hKey1, hKey2, hKey3, hKey4, hKeyTmp;
    TSS_HKEY hSRK;
    TSS_RESULT result;
    TSS_HPOLICY srkUsagePolicy, keyUsagePolicy;
    TSS_HPOLICY hPolicy, tpmUsagePolicy;

    print_begin_test(nameOfFunction);
    printf("KEY structure:\n%s\n", key_structure);//全局字符串 展示密钥结构

    //Create Context
    result = Tspi_Context_Create(&hContext);//获得上下文环境
    if (result != TSS_SUCCESS) {
        print_error("Tspi_Context_Create", result);
        print_error_exit(nameOfFunction, err_string(result));
        exit(result);
    }
    //Connect Context
```

```

result = Tspi_Context_Connect(hContext, get_server(GLOBALSERVER));//连接上下文环境
if (result != TSS_SUCCESS) {
    print_error("Tspi_Context_Connect", result);
    print_error_exit(nameOfFunction, err_string(result));
    Tspi_Context_Close(hContext);
    exit(result);
}

//Load Key By UUID
result = Tspi_Context_LoadKeyByUUID(hContext,
    TSS_PS_TYPE_SYSTEM, SRK_UUID,
    &hSRK);//加载 SRK, 获得 SRK 的 UUID, 存于 hSRK
if (result != TSS_SUCCESS) {
    print_error("Tspi_Context_LoadKeyByUUID", result);
    print_error_exit(nameOfFunction, err_string(result));
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}
#endif TESTSUITE_NOAUTH_SRK
//Get Policy Object
result = Tspi_GetPolicyObject(hSRK, TSS_POLICY_USAGE, &srkUsagePolicy);//获得 SRK 策略, 存于 srkUsagePolicy
if (result != TSS_SUCCESS) {
    print_error("Tspi_GetPolicyObject", result);
    print_error_exit(nameOfFunction, err_string(result));
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}
//Set Secret
result = Tspi_Policy_SetSecret(srkUsagePolicy, //设置 SRK 的授权密钥
    TESTSUITE_SRK_SECRET_MODE,
    TESTSUITE_SRK_SECRET_LEN,
    TESTSUITE_SRK_SECRET);
if (result != TSS_SUCCESS) {
    print_error("Tspi_Policy_SetSecret", result);
    print_error_exit(nameOfFunction, err_string(result));
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}
#endif
// fist unregister keys 注销密钥
Tspi_Context_UnregisterKey( hContext, TSS_PS_TYPE_SYSTEM,
    UUID_K1,
    &hKeyTmp );
Tspi_Context_UnregisterKey( hContext, TSS_PS_TYPE_SYSTEM,
    UUID_K2,
    &hKeyTmp );
Tspi_Context_UnregisterKey( hContext, TSS_PS_TYPE_SYSTEM,
    UUID_K3,
    &hKeyTmp );
Tspi_Context_UnregisterKey( hContext, TSS_PS_TYPE_SYSTEM,
    UUID_K4,
    &hKeyTmp );

//Create K1, parent key is SRK
printf("Create UserK1 and register it to disk.\n");
initFlags = TSS_KEY_TYPE_STORAGE | TSS_KEY_SIZE_2048 |
    TSS_KEY_VOLATILE | TSS_KEY_AUTHORIZATION |
    TSS_KEY_NOT_MIGRATABLE;//重要 存储密钥, 2048 位, 可变密钥, 授权密钥, 不可迁移密钥

result = my_create_load_key(hContext, initFlags, hSRK, &hKey1, "K1");//创建 K1, 上下文, 标志位, 父密钥, 存于哪
里, 名字
if (result != TSS_SUCCESS) {
    print_error("create_key", result);

```

```

    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}

// Register K1
result = Tspi_Context_RegisterKey(hContext, hKey1,
                                TSS_PS_TYPE_SYSTEM,
                                UUID_K1,
                                TSS_PS_TYPE_SYSTEM,
                                SRK_UUID); //注册密钥，上下文，存于哪里，SYSTEM 参数，UUID，SYSTEM 参数，
父密钥的 UUID
if (result != TSS_SUCCESS) {
    print_error("Tspi_Context_RegisterKey", result);
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}
printf("Create and register K1 succeeded!\n");
// Create K2, parent key is K1
printf("Create UserK2 and register it to disk.\n");
initFlags = TSS_KEY_TYPE_SIGNING | TSS_KEY_SIZE_2048 |
            TSS_KEY_VOLATILE | TSS_KEY_AUTHORIZATION |
            TSS_KEY_NOT_MIGRATABLE;
result = my_create_load_key(hContext, initFlags, hKey1, &hKey2, "K2");
if (result != TSS_SUCCESS) {
    print_error("create_key", result);
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}
result = Tspi_Context_RegisterKey(hContext, hKey2,
                                TSS_PS_TYPE_SYSTEM,
                                UUID_K2,
                                TSS_PS_TYPE_SYSTEM,
                                UUID_K1);

if (result != TSS_SUCCESS) {
    print_error("Tspi_Context_RegisterKey", result);
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}
printf("Create and register K2 succeeded!\n");
// K3 , migratable, parent key is SRK
printf("Create UserK3 and register it to disk.\n");
initFlags = TSS_KEY_TYPE_STORAGE | TSS_KEY_SIZE_2048 |
            TSS_KEY_VOLATILE | TSS_KEY_AUTHORIZATION |
            TSS_KEY_MIGRATABLE;
result = my_create_load_key(hContext, initFlags, hSRK, &hKey3, "K3");
if (result != TSS_SUCCESS) {
    print_error("create_key", result);
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}
result = Tspi_Context_RegisterKey(hContext, hKey3,
                                TSS_PS_TYPE_SYSTEM,
                                UUID_K3,
                                TSS_PS_TYPE_SYSTEM,
                                SRK_UUID);

if (result != TSS_SUCCESS) {
    print_error("Tspi_Context_RegisterKey", result);
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}
printf("Create and register K3 succeeded!\n");

```

```

// K4 , migratable , parent key is K3
// TODO:
// 这部分由同学们来完成
//
printf("Create UserK4 and register it to disk.\n");
initFlags = TSS_KEY_TYPE_BIND | TSS_KEY_SIZE_2048 |
            TSS_KEY_VOLATILE | TSS_KEY_AUTHORIZATION |
            TSS_KEY_MIGRATABLE; //绑定密钥, 2048 位, 可变密钥, 授权密钥, 可迁移密钥
result = my_create_load_key(hContext, initFlags, hKey3, &hKey4, "K4"); //创建 K4, 上下文, 标志位, 父密钥, 存于哪
里, 名字
if (result != TSS_SUCCESS) {
    print_error("create_key", result);
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}
result = Tspi_Context_RegisterKey(hContext, hKey4,
                                TSS_PS_TYPE_SYSTEM,
                                UUID_K4,
                                TSS_PS_TYPE_SYSTEM,
                                UUID_K3); //注册密钥, 上下文, 存于哪里, SYSTEM 参数, UUID, SYSTEM 参数,
父密钥的 UUID
if (result != TSS_SUCCESS) {
    print_error("Tspi_Context_RegisterKey", result);
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}

printf("Create and register K4 succeeded!\n");

// Unload keys, pay attention to the order, child key must unload first
result = Tspi_Key_UnloadKey( hKey2 );
result = Tspi_Key_UnloadKey( hKey1 );
result = Tspi_Key_UnloadKey( hKey4 );
result = Tspi_Key_UnloadKey( hKey3 );

if (result != TSS_SUCCESS) {
    if (!checkNonAPI(result)) {
        print_error(nameOfFunction, result);
        print_end_test(nameOfFunction);
        Tspi_Context_FreeMemory(hContext, NULL);
        Tspi_Context_Close(hContext);
        exit(result);
    } else {
        print_error_nonapi(nameOfFunction, result);
        print_end_test(nameOfFunction);
        Tspi_Context_FreeMemory(hContext, NULL);
        Tspi_Context_Close(hContext);
        exit(result);
    }
} else {
    print_success(nameOfFunction, result);
    print_end_test(nameOfFunction);
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(0);
}
}

/* create a key off the parentKey */
TSS_RESULT my_create_key(TSS_HCONTEXT hContext, TSS_FLAG initFlags, TSS_HKEY hParent, TSS_HKEY *hKey,
const char *pName)
{
    TSS_RESULT result;

```

```

TSS_HPOLICY hPolicy, hPolicyMigrate;
char *prompt[256];

//Create Object
result = Tspi_Context_CreateObject(hContext, TSS_OBJECT_TYPE_RSAKEY, initFlags, hKey);
if (result != TSS_SUCCESS) {
    print_error("Tspi_Context_CreateObject", result);
    return(result);
}

if (initFlags & TSS_KEY_AUTHORIZATION) { //设置授权策略
    //printf("Setting secret...\n");
    sprintf(prompt, "Input %s's Usage Pin", pName);
    if ((result = set_popup_secret(hContext, *hKey, TSS_POLICY_USAGE, prompt, 1)))
        return result;
    /*if ((result = set_secret(hContext, *hKey, &hPolicy)))
        return result;*/
}
if (initFlags & TSS_KEY_MIGRATABLE) //设置迁移策略
{
    printf("\n^migratable\n");
    sprintf(prompt, "%s's Migration Pin", pName);
    if ((result = set_popup_secret(hContext, *hKey, TSS_POLICY_MIGRATION, prompt, 1)))
        return result;
}

//CreateKey
result = Tspi_Key_CreateKey(*hKey, hParent, 0);
if (result != TSS_SUCCESS) {
    print_error("Tspi_Key_CreateKey", result);
    return(result);
}

return TSS_SUCCESS;
}

/* create and load a key off the parentKey */
TSS_RESULT
my_create_load_key(TSS_HCONTEXT hContext, TSS_FLAG initFlags,
    TSS_HKEY hParent, TSS_HKEY *hKey, const char *pName)
{
    TSS_RESULT result;

    if ((result = my_create_key(hContext, initFlags, hParent, hKey, pName)))
        return result;

    result = Tspi_Key_LoadKey(*hKey, hParent);
    if (result != TSS_SUCCESS) {
        print_error("Tspi_Key_LoadKey", result);
        return(result);
    }

    return TSS_SUCCESS;
}

```

5.2 加载密钥 load_key.c

```

#include "common.h"

TSS_UUID UUID_K1 = {0, 0, 0, 0, 0, {8, 0, 0, 0, 0, 1}};
TSS_UUID UUID_K2 = {0, 0, 0, 0, 0, {8, 0, 0, 0, 0, 2}};
TSS_UUID UUID_K3 = {0, 0, 0, 0, 0, {8, 0, 0, 0, 0, 3}};
TSS_UUID UUID_K4 = {0, 0, 0, 0, 0, {8, 0, 0, 0, 0, 4}};

```

```

char *key_structure = "SRK\n"
                    "|__UserK1(Storage key, unmigratable)\n"
                    "|  |__UserK2(Signing key, unmigratable)\n"
                    "|\n"
                    "    |__UserK3(Storage key, migratable)\n"
                    "    |  |__UserK4(Bind key, migratable)\n";

int main(int argc, char **argv)
{
    char    version;

    version = parseArgs( argc, argv );
    if (version == TESTSUITE_TEST_TSS_1_2)
        main_v1_2(version);
    //else if (version)
        //main_v1_1();
    else
        print_wrongVersion();
}

main_v1_2(char version){

    char    *nameOfFunction = "Load KEY";
    TSS_HCONTEXT    hContext;
    TSS_HTPM        hTPM;
    TSS_FLAG        initFlags;
    TSS_HKEY        hKey1, hKey2, hKey3, hKey4;
    TSS_HKEY        hSRK;
    TSS_RESULT      result;
    TSS_UUID        uuid;
    TSS_HPOLICY     srkUsagePolicy, keyUsagePolicy, keyMigPolicy, hKeyPolicy;

    print_begin_test(nameOfFunction);
    printf("KEY structure:\n%s\n", key_structure);

    //Create Context
    result = Tspi_Context_Create(&hContext);
    if (result != TSS_SUCCESS) {
        print_error("Tspi_Context_Connect", result);
        print_error_exit(nameOfFunction, err_string(result));
        exit(result);
    }
    //Connect Context
    result = Tspi_Context_Connect(hContext, get_server(GLOBALSERVER));
    if (result != TSS_SUCCESS) {
        print_error("Tspi_Context_Connect", result);
        print_error_exit(nameOfFunction, err_string(result));
        Tspi_Context_Close(hContext);
        exit(result);
    }

    //Load Key By UUID
    result = Tspi_Context_LoadKeyByUUID(hContext,
        TSS_PS_TYPE_SYSTEM, SRK_UUID, &hSRK);
    if (result != TSS_SUCCESS) {
        print_error("Tspi_Context_LoadKeyByUUID", result);
        print_error_exit(nameOfFunction, err_string(result));
        Tspi_Context_Close(hContext);
        exit(result);
    }
    printf("Load SRK succeeded!\n");
#ifdef TESTSUITE_NOAUTH_SRK
    //Get Policy Object
    result = Tspi_GetPolicyObject(hSRK, TSS_POLICY_USAGE, &srkUsagePolicy);
    if (result != TSS_SUCCESS) {
        print_error("Tspi_GetPolicyObject", result);

```

```

        print_error_exit(nameOfFunction, err_string(result));
        Tspi_Context_Close(hContext);
        exit(result);
    }

    //Set Secret
    result = Tspi_Policy_SetSecret(srkUsagePolicy, TESTSUITE_SRK_SECRET_MODE,
        TESTSUITE_SRK_SECRET_LEN, TESTSUITE_SRK_SECRET);
    if (result != TSS_SUCCESS) {
        print_error("Tspi_Policy_SetSecret", result);
        print_error_exit(nameOfFunction, err_string(result));
        Tspi_Context_Close(hContext);
        exit(result);
    }
#endif

// ----- load k1 -----
printf("Loading K1...\n");
result = Tspi_Context_GetKeyByUUID(hContext,
    TSS_PS_TYPE_SYSTEM, UUID_K1,
    &hKey1);
if (result != TSS_SUCCESS) {
    print_error("Tspi_Context_LoadKeyByUUID", result);
    print_error_exit(nameOfFunction, err_string(result));
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}

//验证使用测略
result = Tspi_Key_LoadKey(hKey1, hSRK);
if (result != TSS_SUCCESS) {
    print_error("Tspi_Key_LoadKey", result);
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}
printf("Load UserK1 succeeded!\n");

// ----- load k2 -----
//Load Key By UUID --K2
printf("Loading K2...\n");
result = Tspi_Context_GetKeyByUUID(hContext, TSS_PS_TYPE_SYSTEM,
    UUID_K2,
    &hKey2);
if (result != TSS_SUCCESS) {
    print_error("Tspi_Context_LoadKeyByUUID", result);
    print_error_exit(nameOfFunction, err_string(result));
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}

//验证使用测略
result = set_popup_secret(hContext, hKey1, TSS_POLICY_USAGE, "Input K1's pin", 0);
if (TSS_SUCCESS != result) {
    print_error("set_popup_secret", result);
    Tspi_Context_Close(hContext);
    return result;
}

result = Tspi_Key_LoadKey(hKey2, hKey1);
if (result != TSS_SUCCESS) {
    print_error("Tspi_Key_LoadKey", result);
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}
printf("Load UserK2 succeeded!\n");

```

```

// 给 SRK 授权
#ifdef TESTSUITE_NOAUTH_SRK
//Get Policy Object
result = Tspi_GetPolicyObject(hSRK, TSS_POLICY_USAGE, &srkUsagePolicy);
if (result != TSS_SUCCESS) {
    print_error("Tspi_GetPolicyObject", result);
    print_error_exit(nameOfFunction, err_string(result));
    Tspi_Context_Close(hContext);
    exit(result);
}
//Set Secret
result = Tspi_Policy_SetSecret(srkUsagePolicy, TESTSUITE_SRK_SECRET_MODE,
    TESTSUITE_SRK_SECRET_LEN, TESTSUITE_SRK_SECRET);
if (result != TSS_SUCCESS) {
    print_error("Tspi_Policy_SetSecret", result);
    print_error_exit(nameOfFunction, err_string(result));
    Tspi_Context_Close(hContext);
    exit(result);
}
#endif

// ----- load k3 -----
printf("Loading K3...\n");
result = Tspi_Context_LoadKeyByUUID(hContext,
    TSS_PS_TYPE_SYSTEM, UUID_K3,
    &hKey3);
if (result != TSS_SUCCESS) {
    print_error("Tspi_Context_GetKeyByUUID", result);
    print_error_exit(nameOfFunction, err_string(result));
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}

printf("Load UserK3 succeeded!\n");

// ----- load k4 -----
printf("Loading K4...\n");
// TODO:
//      这部分代码由同学们自己完成
//

result = Tspi_Context_GetKeyByUUID(hContext, TSS_PS_TYPE_SYSTEM,
    UUID_K4,
    &hKey4);
if (result != TSS_SUCCESS) {
    print_error("Tspi_Context_LoadKeyByUUID", result);
    print_error_exit(nameOfFunction, err_string(result));
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(result);
}

printf("Load UserK4 succeeded!\n");
//验证使用测略
result = set_popup_secret(hContext, hKey3, TSS_POLICY_USAGE, "Input K3's pin", 0);
if (TSS_SUCCESS != result) {
    print_error("set_popup_secret", result);
    Tspi_Context_Close(hContext);
    return result;
}
result = Tspi_Key_LoadKey(hKey4, hKey3);
if (result != TSS_SUCCESS) {
    print_error("Tspi_Key_LoadKey", result);
    Tspi_Context_FreeMemory(hContext, NULL);
}

```

```

        Tspi_Context_Close(hContext);
        exit(result);
    }
    printf("Load UserK4 sucesed!\n");
    print_success(nameOfFunction, result);
    print_end_test(nameOfFunction);
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(0);
}

```

5.3 Unseal、解密文件 unseal_file.c

```

#include "common.h"
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <openssl/evp.h>
#include <openssl/aes.h>

int aes_init(unsigned char *key_data, int key_data_len, unsigned char *salt, EVP_CIPHER_CTX *e_ctx)
{
    int i, nrounds = 5;
    unsigned char key[32], iv[32];
    i = EVP_BytesToKey(EVP_aes_256_cbc(), EVP_sha1(), salt, key_data, key_data_len, nrounds, key, iv);
    if (i != 32) {
        printf("Key size is %d bits - should be 256 bits\n", i);
        return -1;
    }
    EVP_CIPHER_CTX_init(e_ctx);
    EVP_DecryptInit_ex(e_ctx, EVP_aes_256_cbc(), NULL, key, iv);
    return 0;
}

unsigned char *aes_decrypt(EVP_CIPHER_CTX *e, unsigned char *plaintext, int *len)
{
    int c_len = *len + AES_BLOCK_SIZE, f_len = 0;
    unsigned char *ciphertext = malloc(c_len);
    EVP_DecryptInit_ex(e, NULL, NULL, NULL, NULL);
    EVP_DecryptUpdate(e, ciphertext, &c_len, plaintext, *len);
    EVP_DecryptFinal_ex(e, ciphertext+c_len, &f_len);
    *len = c_len + f_len;
    return ciphertext;
}

void usage(char *pch_name)
{
    printf("Usage: %s source destination\n", pch_name);
    printf("eg: %s plaintext_file ciphertext_file\n", pch_name);
}

int main(int argc, char **argv)
{
    #define BUF_LEN    (1024*1024)
    #define KEY_SIZE 64
    TSS_RESULT result;
    TSS_HCONTEXT hContext;
    TSS_HKEY hSRK, hKey;
    TSS_HPOLICY hPolicy;
    TSS_HTPM hTPM;
    TSS_HENCDATA hEncData;
    TSS_HPCRS hPCrs;
    UINT32 DataSealedLen, DataUnsealedLen; //DataSealedLen 是 Seal 的密钥长度从文件读取，
    DataUnsealedLen 是解封之后密钥的长度
    BYTE *DataUnsealed, *DataSealed, *dataBuf; //DataUnsealed 就是对称密钥，DataSealed 是从文件读取的

```

```

FILE *fpIn = NULL, *fpOut = NULL;
int len, size; //len 是 fread 读取到的长度, size 是文件实际大小
char *pBufIn = NULL, *pBufOut = NULL;
unsigned int salt[] = {12345, 54321};
EVP_CIPHER_CTX de;
TSS_UUID UUID_K1 = {0, 0, 0, 0, 0, {8, 0, 0, 0, 0, 1}};

if (argc < 3) {
    usage(argv[0]);
    return 0;
}

result = Tspi_Context_Create(&hContext);
if (TSS_SUCCESS != result) {
    print_error("Tspi_Context_Create", result);
    Tspi_Context_Close(hContext);
    return result;
}

result = Tspi_Context_Connect(hContext, get_server(GLOBALSERVER));
if (TSS_SUCCESS != result) {
    print_error("Tspi_Context_Connect", result);
    Tspi_Context_Close(hContext);
    return result;
}

result = Tspi_Context_GetTpmObject(hContext, &hTPM);
if (TSS_SUCCESS != result) {
    print_error("Tspi_Context_GetTpmObject", result);
    Tspi_Context_Close(hContext);
    return result;
}

result = Tspi_Context_LoadKeyByUUID(hContext,
                                     TSS_PS_TYPE_SYSTEM,
                                     SRK_UUID,
                                     &hSRK);
if (TSS_SUCCESS != result) {
    print_error("Tspi_Context_LoadKeyByUUID", result);
    Tspi_Context_Close(hContext);
    return result;
}

#ifdef TESTSUITE_NOAUTH_SRK
result = Tspi_GetPolicyObject(hSRK, TSS_POLICY_USAGE, &hPolicy);
if (TSS_SUCCESS != result) {
    print_error("Tspi_GetPolicyObject", result);
    Tspi_Context_Close(hContext);
    return result;
}

result = Tspi_Policy_SetSecret(hPolicy,
                              TESTSUITE_SRK_SECRET_MODE,
                              TESTSUITE_SRK_SECRET_LEN,
                              TESTSUITE_SRK_SECRET);
if (TSS_SUCCESS != result) {
    print_error("Tspi_Policy_SetSecret", result);
    Tspi_Context_Close(hContext);
    return result;
}
#endif // #ifndef TESTSUITE_NOAUTH_SRK

result = Tspi_Context_CreateObject(hContext,
                                   TSS_OBJECT_TYPE_PCRS,
                                   0,
                                   &hPcrs);
if (TSS_SUCCESS != result) {

```

```

        print_error("Tspi_Context_CreateObject", result);
        Tspi_Context_Close(hContext);
        return result;
    }

    result = Tspi_Context_CreateObject(hContext,
                                      TSS_OBJECT_TYPE_ENCDATA,
                                      TSS_ENCDATA_SEAL,
                                      &hEncData);
    if (TSS_SUCCESS != result) {
        print_error("Tspi_Context_CreateObject", result);
        Tspi_Context_Close(hContext);
        return result;
    }

    result = set_secret(hContext, hEncData, &hPolicy);
    if (TSS_SUCCESS != result) {
        print_error("set_secret", result);
        Tspi_Context_Close(hContext);
        return result;
    }

    result = Tspi_Context_LoadKeyByUUID(hContext,
                                       TSS_PS_TYPE_SYSTEM,
                                       UUID_K1,
                                       &hKey);
    if (TSS_SUCCESS != result) {
        print_error("Tspi_Context_LoadKeyByUUID", result);
        Tspi_Context_Close(hContext);
        return -1;
    }

    result = set_popup_secret(hContext,
                             hKey,
                             TSS_POLICY_USAGE,
                             "Input K1's Pin\n",
                             0);
    if (TSS_SUCCESS != result) {
        print_error("set_popup_secret", result);
        Tspi_Context_Close(hContext);
        return result;
    }
}

/***** 读文件 获得相关数据 *****/
fpIn = fopen(argv[1], "rb");
if (!fpIn) {
    printf("open %s failed\n", argv[1]);
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(-1);
}

if (fread(&DataSealedLen, 1, sizeof(UINT32), fpIn) != sizeof(UINT32)) //获得 Sealed 的密钥长度
{
    printf("fread failed in lenth of sealed data\n");
    fclose(fpIn);
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
    exit(-1);
}

DataSealed = (BYTE *)malloc(DataSealedLen);
if (fread(DataSealed, 1, DataSealedLen, fpIn) != DataSealedLen) //获得 Sealed 的密钥
{
    printf("fread failed in getting sealed data\n");
    free(DataSealed);
    fclose(fpIn);
    Tspi_Context_FreeMemory(hContext, NULL);
    Tspi_Context_Close(hContext);
}

```

```

        exit(-1);
    }
    if (fread(&size,1,sizeof(int), fpIn) != sizeof(int))//获得文件长度
    {
        printf("fread failed in size of file\n");
        fclose(fpIn);
        free(DataSealed);
        Tspi_Context_FreeMemory(hContext, NULL);
        Tspi_Context_Close(hContext);
        exit(-1);
    }
    pBufIn = (char *)malloc(size);
    if (fread(pBufIn,1,size, fpIn) != size)//获得加密的文件数据
    {
        printf("fread failed getting data of file\n");
        fclose(fpIn);
        free(DataSealed);
        free(pBufIn);
        Tspi_Context_FreeMemory(hContext, NULL);
        Tspi_Context_Close(hContext);
        exit(-1);
    }
    fclose(fpIn);
    /***** 读文件结束 *****/

    result = Tspi_SetAttribData(hEncData,
                                TSS_TSPATTRIB_ENCDATA_BLOB,
                                TSS_TSPATTRIB_ENCDATABLOB_BLOB,
                                DataSealedLen,
                                DataSealed);
    if (TSS_SUCCESS != result) {
        print_error("Tspi_SetAttribData", result);
        free(DataSealed);
        DataSealed = NULL;
        Tspi_Context_FreeMemory(hContext, NULL);
        Tspi_Context_Close(hContext);
        exit(result);
    }

    result = Tspi_Data_Unseal(hEncData,
                              hKey,
                              &DataUnsealedLen,
                              &DataUnsealed);
    printf("Unsealed Data:\n");
    print_hex(DataUnsealed, DataUnsealedLen);//获得了 AES 密钥

    if (aes_init(DataUnsealed, KEY_SIZE, (unsigned char *)&salt, &de)) { //DataUnsealed 是 AES 的密钥, salt
是盐值
        printf("aes_init failed\n");
        Tspi_Context_Close(hContext);
        free(pBufIn);
        return -1;
    }
    pBufOut = (char *)malloc(size);
    pBufOut = aes_decrypt(&de, pBufIn, &size);//pBufOut 是加密文件数据

    /***** 开始写入文件 *****/
    fpOut = fopen(argv[2], "wb");
    if (!fpOut) {
        printf("open file: %s failed\n", argv[2]);
        Tspi_Context_Close(hContext);
        free(pBufIn);
        free(pBufOut);
        return -1;
    }
    len = fwrite(pBufOut, 1, size, fpOut);

```

```

    if (len != size) {
        printf("fwrite failed\n");
        Tspi_Context_Close(hContext);
        free(pBufIn);
        free(pBufOut);
        fclose(fpOut);
        return -1;
    }
    /***** 写文件结束 *****/
    fclose(fpOut);
    free(pBufIn);
    free(pBufOut);

    Tspi_Context_Close(hContext);

    return 0;
}

```

5.4 远程证明 platform_dst.c

```

#define PUB_SRK          "srk.pub"
#define MIG_BLOB        "mig.blob"

#include "common.h"
#include <stdio.h>
#include <string.h>

void usage(char *name)
{
    printf("Usage: %s -g|-m\n", name);
    printf("-g: Generate SRK pubkey and save it to file %s\n", PUB_SRK);
    printf("-m: Migrate key from file %s\n", MIG_BLOB);
}

int
gen_pubSrK(void)
{
    TSS_RESULT result;
    TSS_HCONTEXT hContext;
    TSS_HKEY hSRK;
    TSS_HPOLICY hPolicy;
    TSS_HTPM hTPM;
    UINT32 u32PubSrKLen;
    BYTE *prgbPubSrK;
    FILE *fp;
    size_t len;

    // Create Context
    result = Tspi_Context_Create(&hContext);
    if (TSS_SUCCESS != result) {
        print_error("Tspi_Context_Create", result);
        return result;
    }

    // Connect Context
    result = Tspi_Context_Connect(hContext, NULL);
    if (TSS_SUCCESS != result) {
        print_error("Tspi_Context_Connect", result);
        Tspi_Context_Close(hContext);
        return result;
    }

    // Get TPM Object

```

```

result = Tspi_Context_GetTpmObject(hContext, &hTPM);
if (TSS_SUCCESS != result) {
    print_error("Tspi_Context_GetTpmObject", result);
    Tspi_Context_Close(hContext);
    return result;
}

// Set TPM Usage Secret
result = Tspi_GetPolicyObject(hTPM, TSS_POLICY_USAGE, &hPolicy);
if (TSS_SUCCESS != result) {
    print_error("Tspi_GetPolicyObject", result);
    Tspi_Context_Close(hContext);
    return result;
}
result = Tspi_Policy_SetSecret(hPolicy,
    TESTSUITE_OWNER_SECRET_MODE,
    TESTSUITE_OWNER_SECRET_LEN,
    TESTSUITE_OWNER_SECRET);
if (TSS_SUCCESS != result) {
    print_error("Tspi_Policy_SetSecret", result);
    Tspi_Context_Close(hContext);
    return result;
}

// Load SRK
result = Tspi_Context_LoadKeyByUUID(hContext,
    TSS_PS_TYPE_SYSTEM,
    SRK_UUID,
    &hSRK);
if (TSS_SUCCESS != result) {
    print_error("Tspi_Context_LoadKeyByUUID", result);
    Tspi_Context_Close(hContext);
    return result;
}

#ifdef TESTSUITE_NOAUTH_SRK
// Set SRK Usage Secret
result = Tspi_GetPolicyObject(hSRK, TSS_POLICY_USAGE, &hPolicy);
if (TSS_SUCCESS != result) {
    print_error("Tspi_GetPolicyObject", result);
    Tspi_Context_Close(hContext);
    return result;
}

result = Tspi_Policy_SetSecret(hPolicy,
    TESTSUITE_SRK_SECRET_MODE,
    TESTSUITE_SRK_SECRET_LEN,
    TESTSUITE_SRK_SECRET);
if (TSS_SUCCESS != result) {
    print_error("Tspi_Policy_SetSecret", result);
    Tspi_Context_Close(hContext);
    return result;
}
#endif // #ifdef TESTSUITE_NOAUTH_SRK

// Get SRK Pub Key
result = Tspi_TPM_OwnerGetSRKPubKey(hTPM, &u32PubSrLen, &prgbPubSr); //获得 SRK 公钥, 存于
prgbPubSr
if (TSS_SUCCESS != result) {
    print_error("Tspi_TPM_OwnerGetSRKPubKey", result);
    Tspi_Context_Close(hContext);
    return result;
}

fp = fopen(PUB_SRK, "wb");
if (!fp) {
    fprintf(stderr, "open file: %s failed\n", PUB_SRK);
}

```

```

        Tspi_Context_Close(hContext);
        return -1;
    }

    len = fwrite(&u32PubSrkJen, sizeof(u32PubSrkJen), 1, fp);//写入长度
    if (1 != len) {
        fprintf(stderr, "fwrite u32PubSrkJen failed\n");
        fclose(fp);
        Tspi_Context_Close(hContext);
        return -1;
    }

    len = fwrite(prgbPubSrkJ, u32PubSrkJLen, 1, fp);//写入公钥
    if (1 != len) {
        fprintf(stderr, "fwrite prgPubSrkJ failed\n");
        fclose(fp);
        Tspi_Context_Close(hContext);
        return -1;
    }

    fclose(fp);

    Tspi_Context_Close(hContext);

    printf("Generating Pub Key Success\n");
    return 0;
}

int
mig_key(void)
{
    TSS_RESULT result;
    TSS_HCONTEXT hContext;
    TSS_HKEY hSRK;
    TSS_HPOLICY hPolicy;
    TSS_HKEY hNewMigKey;
    FILE *fp;
    size_t len;
    UINT32 u32RandomLen;
    BYTE *pRandom;
    UINT32 u32MigBlobLen;
    BYTE *pMigBlob;

    // Create Context
    result = Tspi_Context_Create(&hContext);
    if (TSS_SUCCESS != result) {
        print_error("Tspi_Context_Create", result);
        return result;
    }

    // Connect Context
    result = Tspi_Context_Connect(hContext, NULL);
    if (TSS_SUCCESS != result) {
        print_error("Tspi_Context_Connect", result);
        Tspi_Context_Close(hContext);
        return result;
    }

    // Load SRK
    result = Tspi_Context_LoadKeyByUUID(hContext,
                                         TSS_PS_TYPE_SYSTEM,
                                         SRK_UUID,
                                         &hSRK);//加载接收方的 SRK
    if (TSS_SUCCESS != result) {
        print_error("Tspi_Context_LoadKeyByUUID", result);
        Tspi_Context_Close(hContext);
    }

```

```

        return result;
    }

#ifndef TESTSUITE_NOAUTH_SRK
    // Set SRK Usage Secret
    result = Tspi_GetPolicyObject(hSRK, TSS_POLICY_USAGE, &hPolicy);
    if (TSS_SUCCESS != result) {
        print_error("Tspi_GetPolicyObject", result);
        Tspi_Context_Close(hContext);
        return result;
    }

    result = Tspi_Policy_SetSecret(hPolicy,
        TESTSUITE_SRK_SECRET_MODE,
        TESTSUITE_SRK_SECRET_LEN,
        TESTSUITE_SRK_SECRET);
    if (TSS_SUCCESS != result) {
        print_error("Tspi_Policy_SetSecret", result);
        Tspi_Context_Close(hContext);
        return result;
    }
}
#endif // #ifndef TESTSUITE_NOAUTH_SRK

// Create New Migrated Key Object
result = Tspi_Context_CreateObject(hContext,
    TSS_OBJECT_TYPE_RSAKEY,
    TSS_KEY_TYPE_SIGNING |
    TSS_KEY_SIZE_2048 |
    TSS_KEY_NO_AUTHORIZATION |
    TSS_KEY_MIGRATABLE,
    &hNewMigKey); //hNewMigKey 准备接受迁移的密钥
if (TSS_SUCCESS != result) {
    print_error("Tspi_Context_CreateObject", result);
    Tspi_Context_Close(hContext);
    return result;
}

// Load Migration Blob
fp = fopen(MIG_BLOB, "rb");
if (!fp) {
    fprintf(stderr, "open file: %s failed\n", MIG_BLOB);
    Tspi_Context_Close(hContext);
    return -1;
}

len = fread(&u32RandomLen, sizeof(UINT32), 1, fp);
if (1 != len) {
    fprintf(stderr, "fread u32RandomLen failed\n");
    fclose(fp);
    Tspi_Context_Close(hContext);
    return -1;
}

pRandom = malloc(u32RandomLen);
if (!pRandom) {
    perror("No Memory\n");
    fclose(fp);
    Tspi_Context_Close(hContext);
    return -1;
}

len = fread(pRandom, u32RandomLen, 1, fp);
if (1 != len) {
    fprintf(stderr, "fread pRandom failed");
    free(pRandom);
    fclose(fp);
    Tspi_Context_Close(hContext);
}

```

```

        return -1;
    }

    len = fread(&u32MigBlobLen, sizeof(UINT32), 1, fp);
    if (1 != len) {
        fprintf(stderr, "fread u32MigBlob failed\n");
        fclose(fp);
        free(pRandom);
        Tspi_Context_Close(hContext);
        return -1;
    }

    pMigBlob = malloc(u32MigBlobLen);
    if (!pMigBlob) {
        perror("No Memory\n");
        fclose(fp);
        free(pRandom);
        Tspi_Context_Close(hContext);
        return -1;
    }

    len = fread(pMigBlob, u32MigBlobLen, 1, fp);
    if (1 != len) {
        fprintf(stderr, "fread pMigBlob\n");
        fclose(fp);
        free(pRandom);
        free(pMigBlob);
        Tspi_Context_Close(hContext);
        return -1;
    }

    fclose(fp);

    // Convert Migration Blob

    // TODO: 完成以下代码, 参考 Tspi_Key_ConvertMigrationBlob
    //从 mig.blob 中已经获取了 u32RandomLen, pRandom,u32MigBlobLen,pMigBlob
    result = Tspi_Key_ConvertMigrationBlob(hNewMigKey,
        hSRK,
        u32RandomLen,
        pRandom,
        u32MigBlobLen,
        pMigBlob);

    free(pRandom);
    free(pMigBlob);
    if (TSS_SUCCESS != result) {
        print_error("Tspi_Key_ConvertMigrationBlob", result);
        Tspi_Context_Close(hContext);
        return result;
    }

    // Load the New Migrated Key
    result = Tspi_Key_LoadKey(hNewMigKey, hSRK);
    if (TSS_SUCCESS != result) {
        print_error("Tspi_Key_Load", result);
        Tspi_Context_Close(hContext);
        return result;
    }

    // verify that the new migrated key is valid
    result = sign_and_verify(hContext, hNewMigKey);
    if (TSS_SUCCESS != result) {
        print_error("sign_and_verify", result);
        Tspi_Context_Close(hContext);
        return result;
    }
}

```

```

    Tspi_Context_Close(hContext);

    printf("Migration Success\n");
    return 0;
}

int
main(int argc, char **argv)
{
    if (argc < 2) {
        usage(argv[0]);
        return 0;
    }

    if (!strcmp("-g", argv[1])) {
        return gen_pubSrk();
    } else if (!strcmp("-m", argv[1])) {
        return mig_key();
    } else {
        usage(argv[0]);
        return 0;
    }
}

```